



# Deep Learning Foundations

**Join the lecture online on *your* dashboard.**

**Let's start with a minute of silence.**

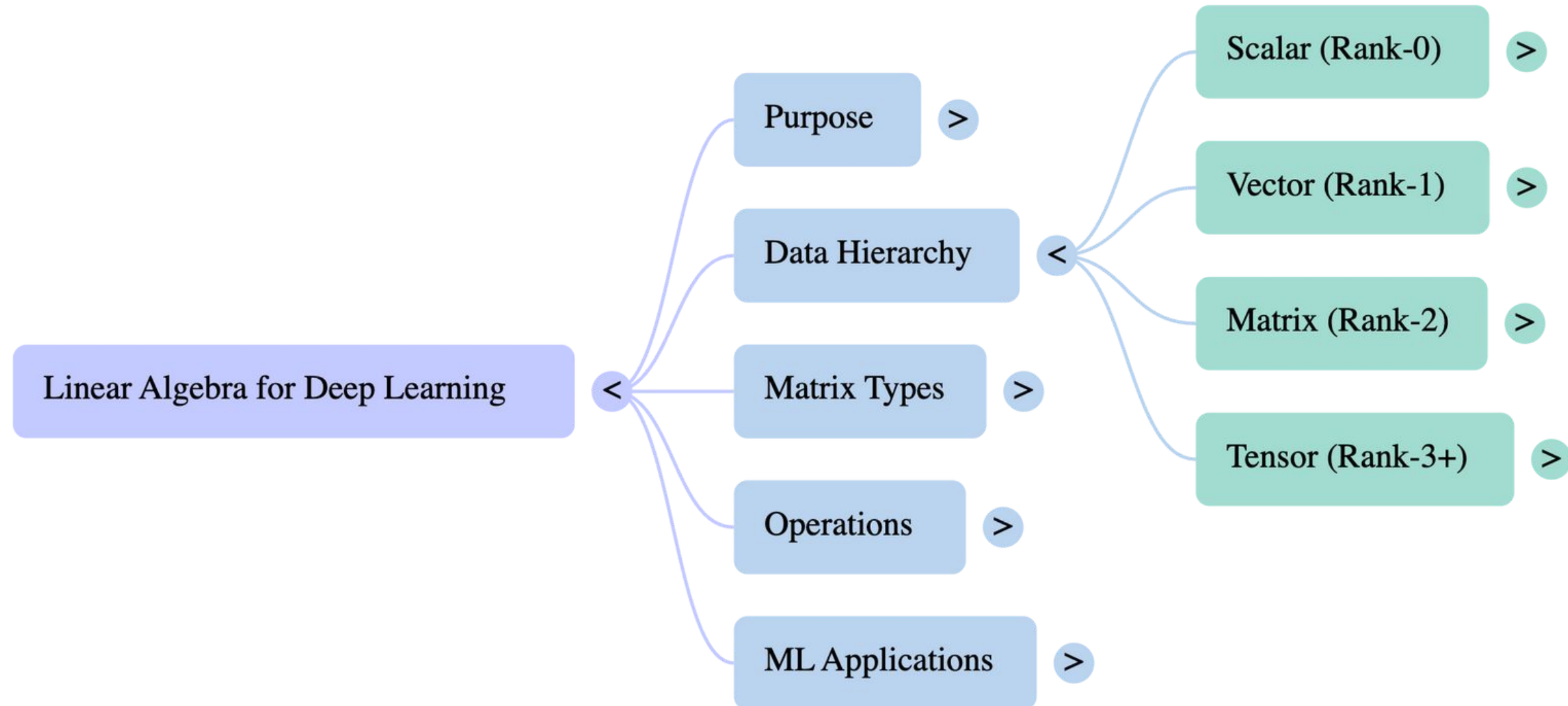
आचार्यात् पादं आधत्ते पादं शिष्यः स्वमेधया ।  
पादं सब्रह्मचारिभ्यः पादं कालक्रमेण च ॥

Meaning:

A student acquires knowledge in four equal parts:

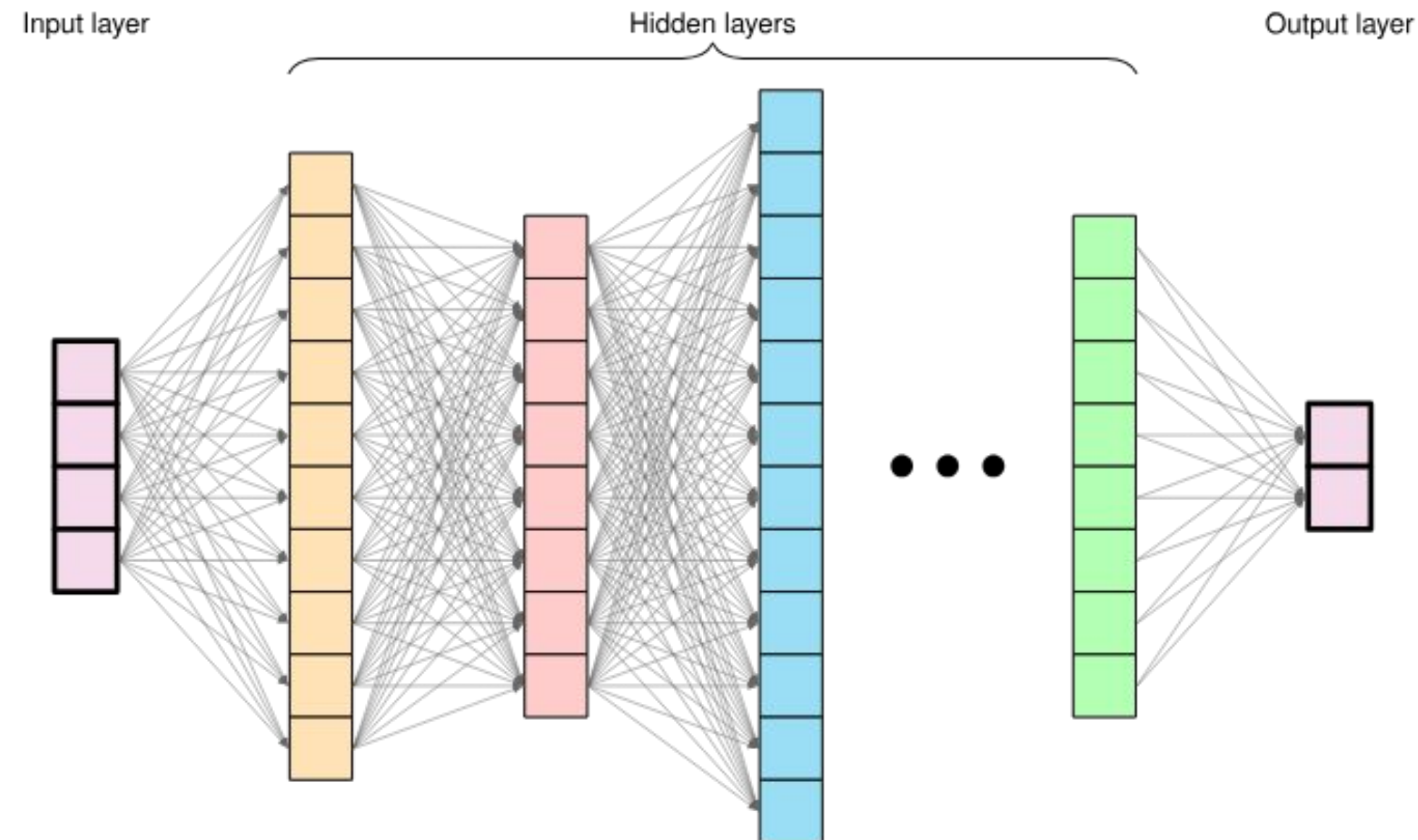
- One-fourth from the teacher
- One-fourth through self-reflection and independent thinking
- One-fourth through discussions with peers and fellow learners
- One-fourth over time through personal experience

# Mind Map





# Foundations of Linear Algebra for Deep Learning



Purpose of Lecture:

To build the mathematical foundation needed to understand how neural networks represent, transform, and learn from data.

# Today's Agenda

## Foundation of Deep Learning:

- Linear Algebra Basics
- Scalars
- Vectors
- Tensors
- Matrices and Operations
- Norms

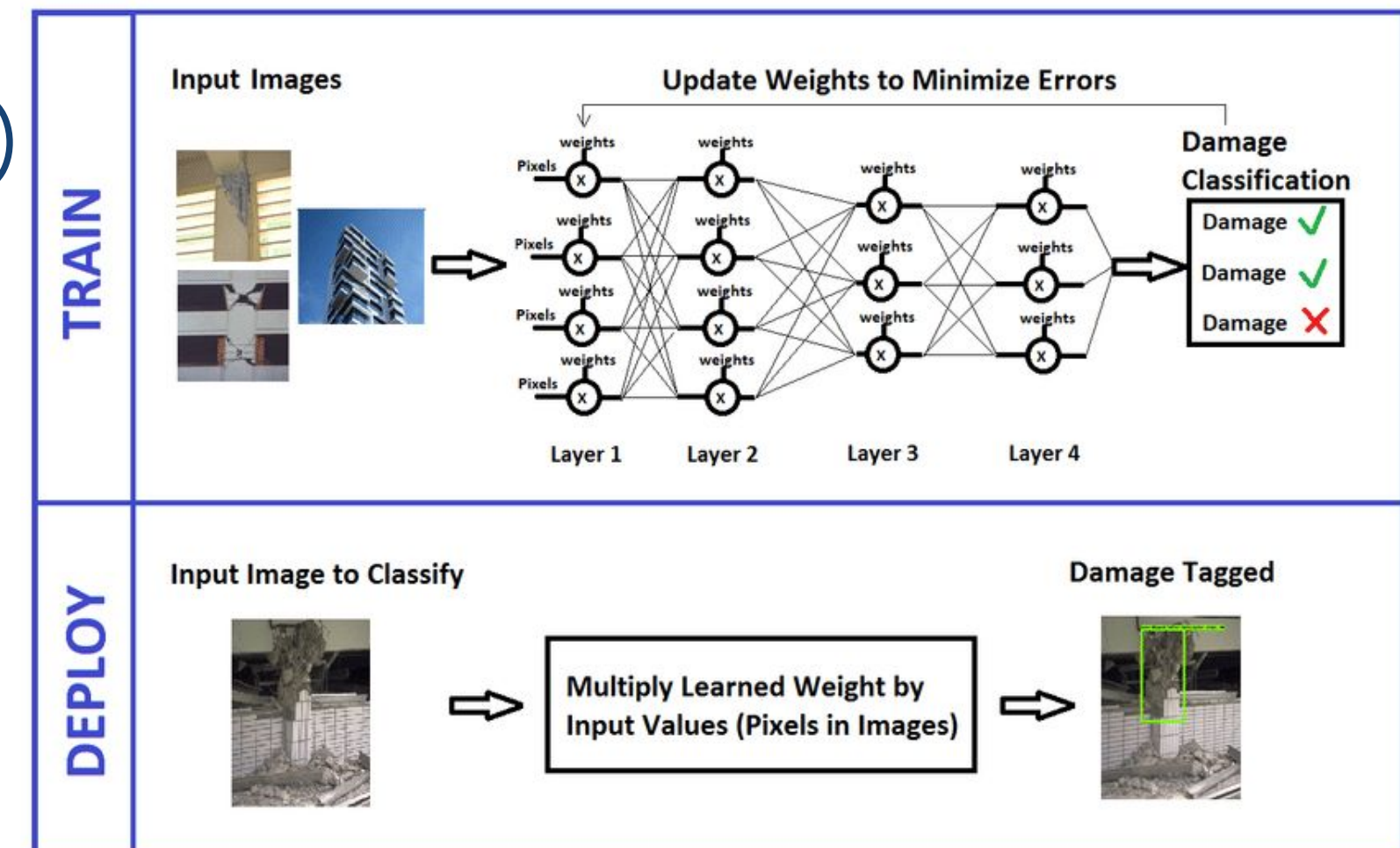


# Why Are We Learning Linear Algebra?

Deep learning = Representation learning  
(automated feature generation)

Every time a neural network:

- Classifies an image
- Generates text
- Recommends a movie
- Translates a sentence
- Predicts a grade or price



...it is performing matrix and vector operations behind the scenes.



# Why Are We Learning Linear Algebra?

Where Linear Algebra Shows Up:

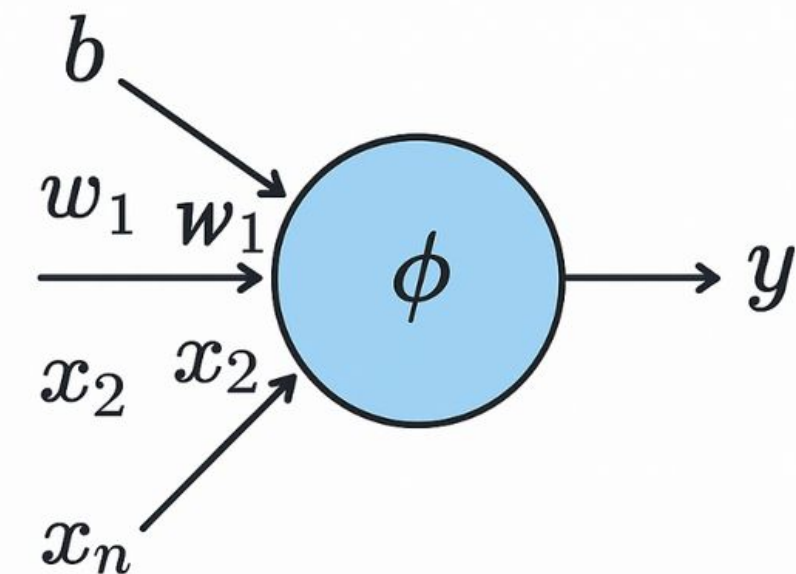
- Weights in neural networks = matrices
- Input features = vectors
- Images = 3D tensors (height  $\times$  width  $\times$  channels)
- Video = 4D tensors
- Word embeddings = high-dimensional vectors
- Batch processing = stacked tensors
- Backpropagation = chain of matrix multiplications

Relatable Analogy:

Think of linear algebra as the grammar and vocabulary of deep learning.

Without the language, you can't write or understand the story.

## Artificial Neuron



# A Day in Deep Learning

Imagine this scenario:



# A Day in Deep Learning

What is actually happening?



# A Day in Deep Learning

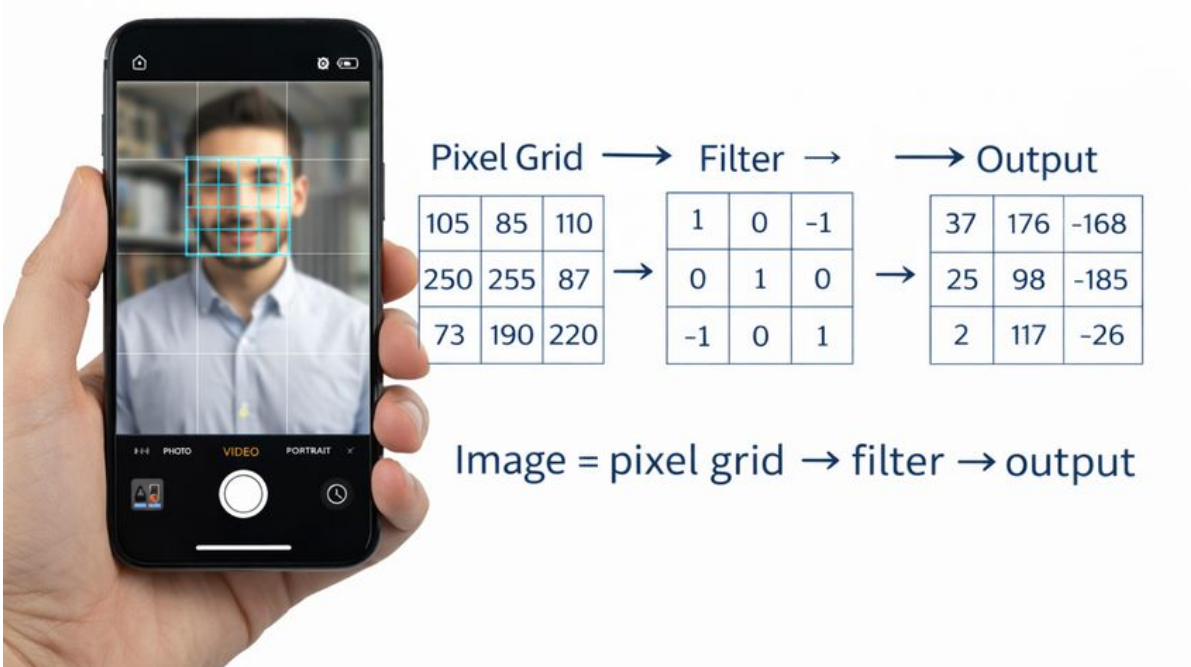
Key Idea:

If you understand vectors, matrices, and tensors...  
you understand the mathematics behind modern AI.

Image Suggestion:

Image = pixel grid → filter → output.  
Or a phone camera UI with a mathematical grid overlay.

| Real-Life Action   | Mathematical Operation   |
|--------------------|--------------------------|
| Your photo         | Tensor of pixel values   |
| Filter applied     | Matrix convolution       |
| Face detection     | Vector–matrix operations |
| Caption suggestion | Embedding + dot product  |
| Auto-brightness    | Matrix scaling           |





# Scalars → Vectors → Matrices → Tensors (The Progression)

Start small and grow:

**Scalar** — 1 number

Example: Temperature = 32°, Learning rate = 0.001

**Vector** — 1D list of numbers

Example: Student profile = [height, weight, age]

**Matrix** — 2D table of numbers

Example: Classroom gradebook (students × assignments)

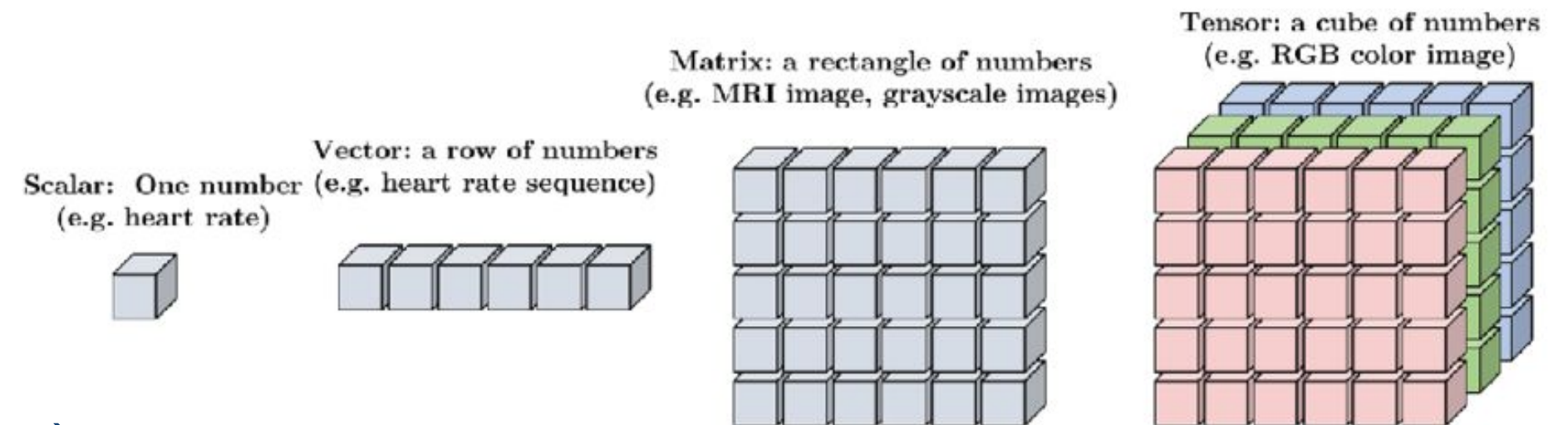
**Tensor** — Multi-dimensional table

Examples:

- RGB image = 3D tensor
- Batch of images = 4D
- Video = 5D

**Why this matters:**

Neural networks consume and produce tensors at every step.



# Scalars

A scalar is a single number.

No direction. No structure. No dimensions.

Examples in Deep Learning:

- Learning rate: 0.001
- Loss value: 2.57
- Bias term:  $b = 3.5$
- Pixel intensity: 255

**Scalar: One number**  
(e.g. heart rate)



Analogy :

A scalar is like a single fact about your day: "Today I studied for 3 hours."

Why Scalars Matter:

- Combine with matrices and vectors
- Represent thresholds, constants, and coefficients



# Vectors

A vector is an ordered list of numbers, representing multiple related quantities.

In Deep Learning Vectors represent:

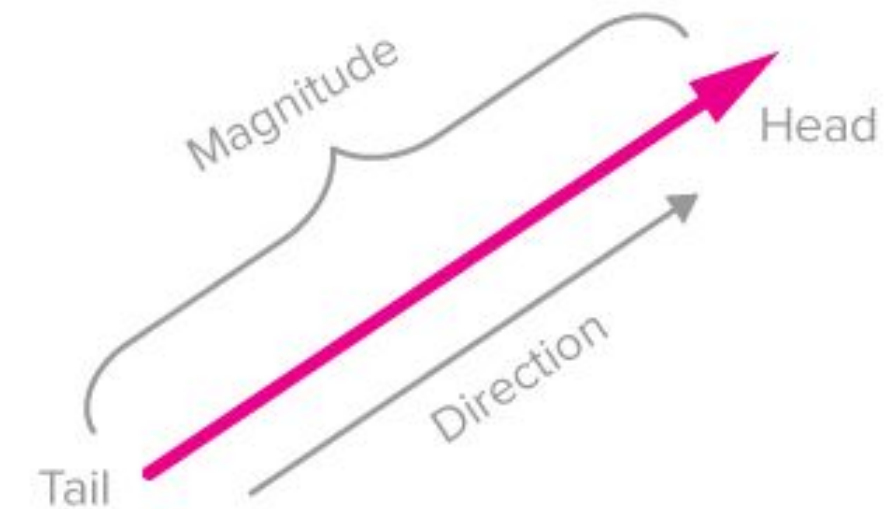
- Features
- Gradients
- Embeddings
- Outputs from a neural layer

Examples

- Your “daily profile” → [sleep\_hours, classes\_attended, meals, study\_hours,
- A movie’s rating → [story\_score, acting\_score, visuals\_score]
- A word embedding (e.g., “apple”) → 300-dimensional vector

A vector has:

- Magnitude (how big?)
- Direction (towards what?)

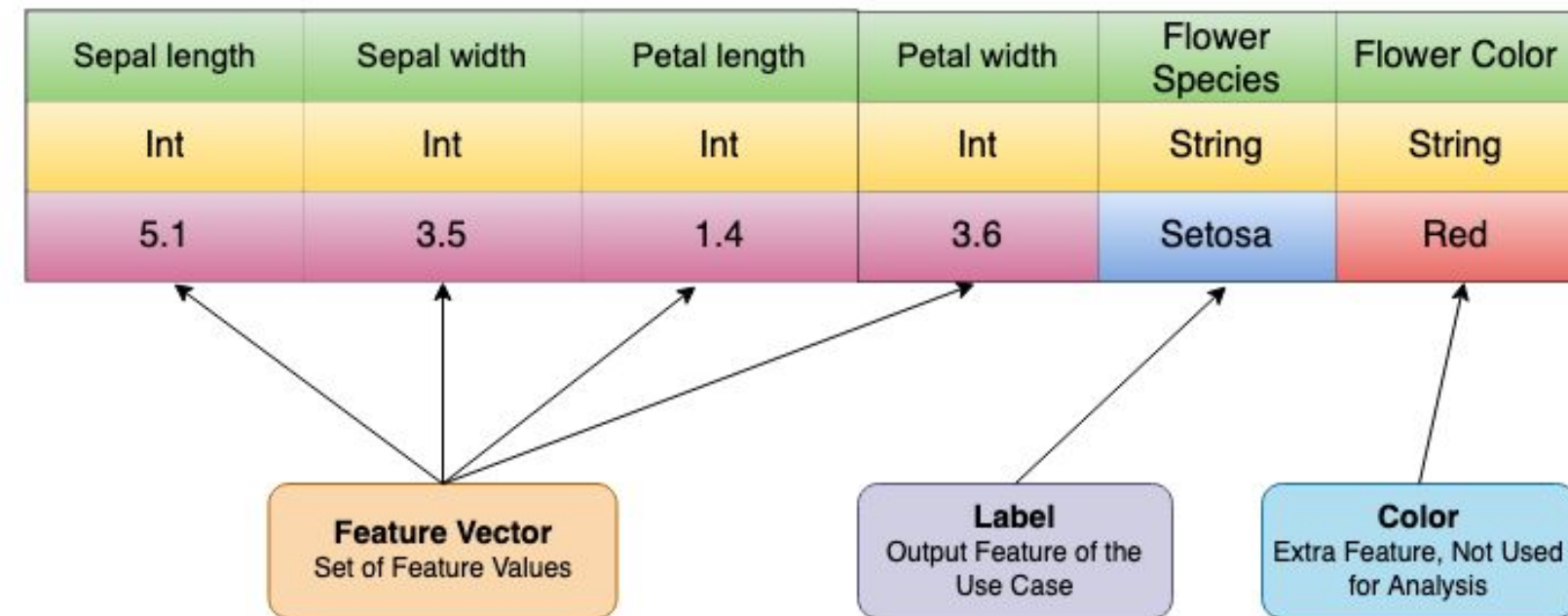


# Vectors

## Vector Examples in ML & Real Life

### In Deep Learning

- Word embeddings → 300D vector
- Image row → 1D vector
- Hidden state in RNN → vector
- Gradients → vector



### Relatable Real-Life Vector Examples:

1. Student performance vector: [marks\_math, marks\_science, marks\_english]
2. A laptop specification vector: [RAM(GB), SSD(GB), processor\_speed(GHz)]
3. A gym workout vector: [reps, sets, weight]

### Intuition:

A vector is like a “bundle” of features.

Each number = one attribute or measurement.

# Dot Product

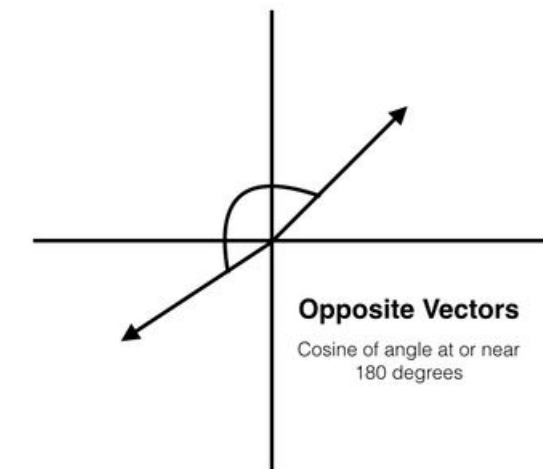
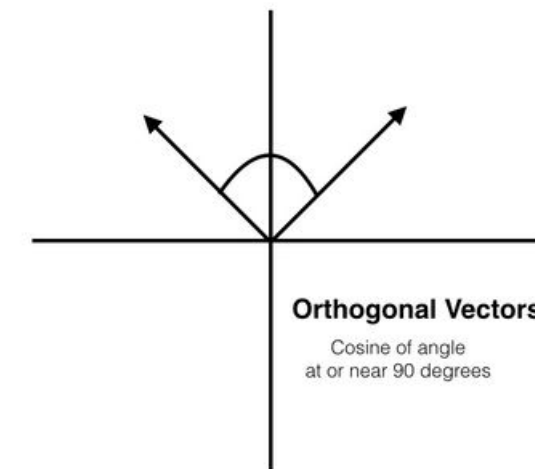
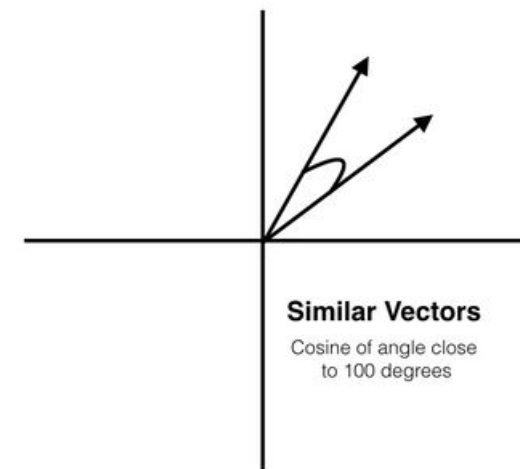
For vectors:

$$x \cdot y = x_1y_1 + x_2y_2 + \dots + x_ny_n$$

## Geometric Meaning

Dot product measures:

- Similarity
- Alignment
- Angle between vectors



## Real-Life Analogies

- Spotify recommendations: similarity between your taste vector & song vector
- Netflix: similarity between user vector & movie vector
- Word embeddings: “king” is similar to “queen” if their vectors align

# Visualizing Dot Product with Angle

## Angle Formula

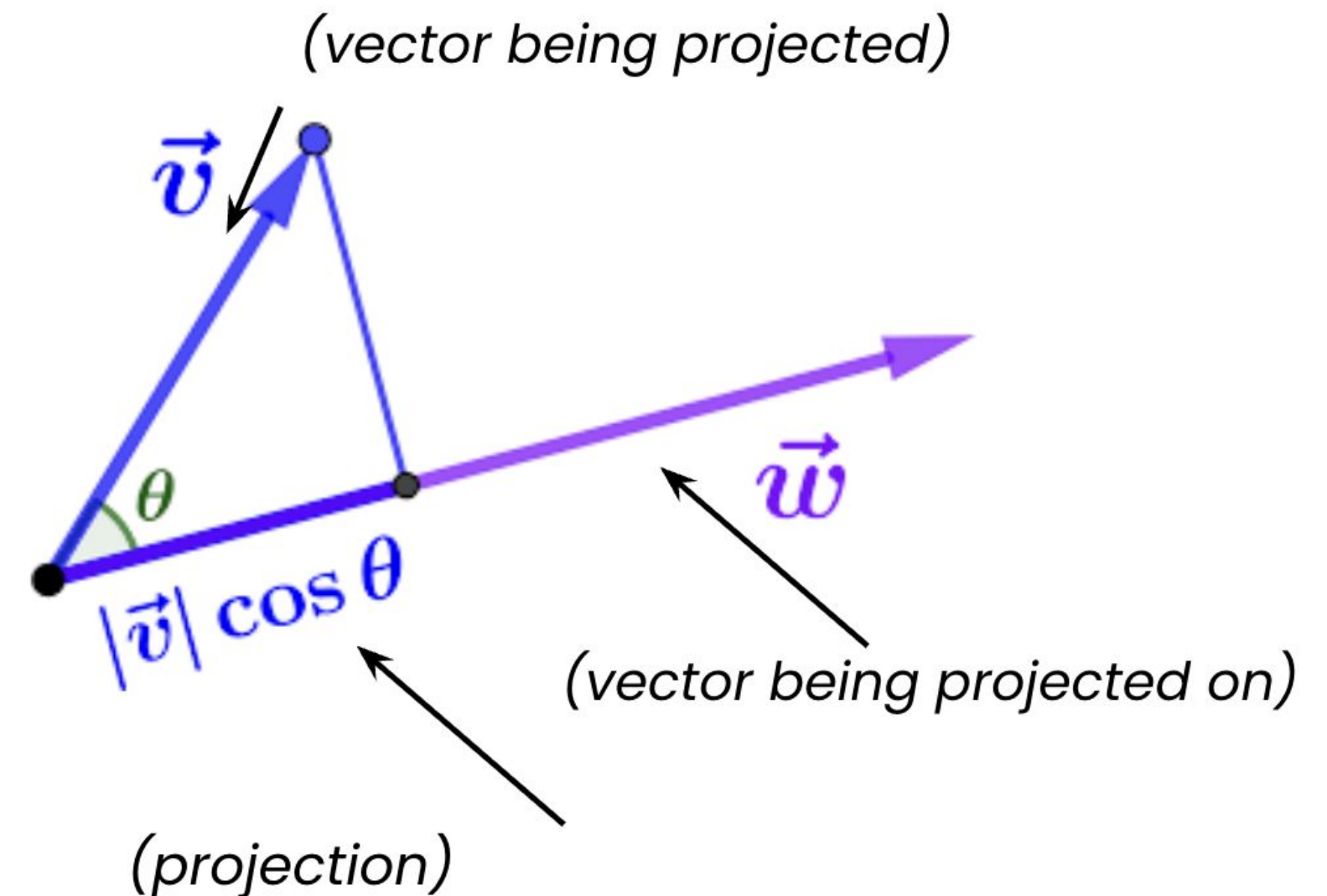
$$\mathbf{V} \cdot \mathbf{W} = \|\mathbf{V}\| \|\mathbf{W}\| \cos(\theta)$$

**Interpretation :**  $\|\mathbf{V}\| \cos \theta$  is a projection of  $\mathbf{v}$  on  $\mathbf{w}$

- If  $\cos(\theta) \approx 1 \rightarrow$  vectors are similar
- If  $\cos(\theta) = 0 \rightarrow$  vectors are orthogonal
- If  $\cos(\theta) < 0 \rightarrow$  vectors point in opposite directions

## How it's used in ML :

- Used in similarity search
- Used in attention mechanisms
- Used in clustering and embedding models
- Shows how "close" two data points are



# Tensors

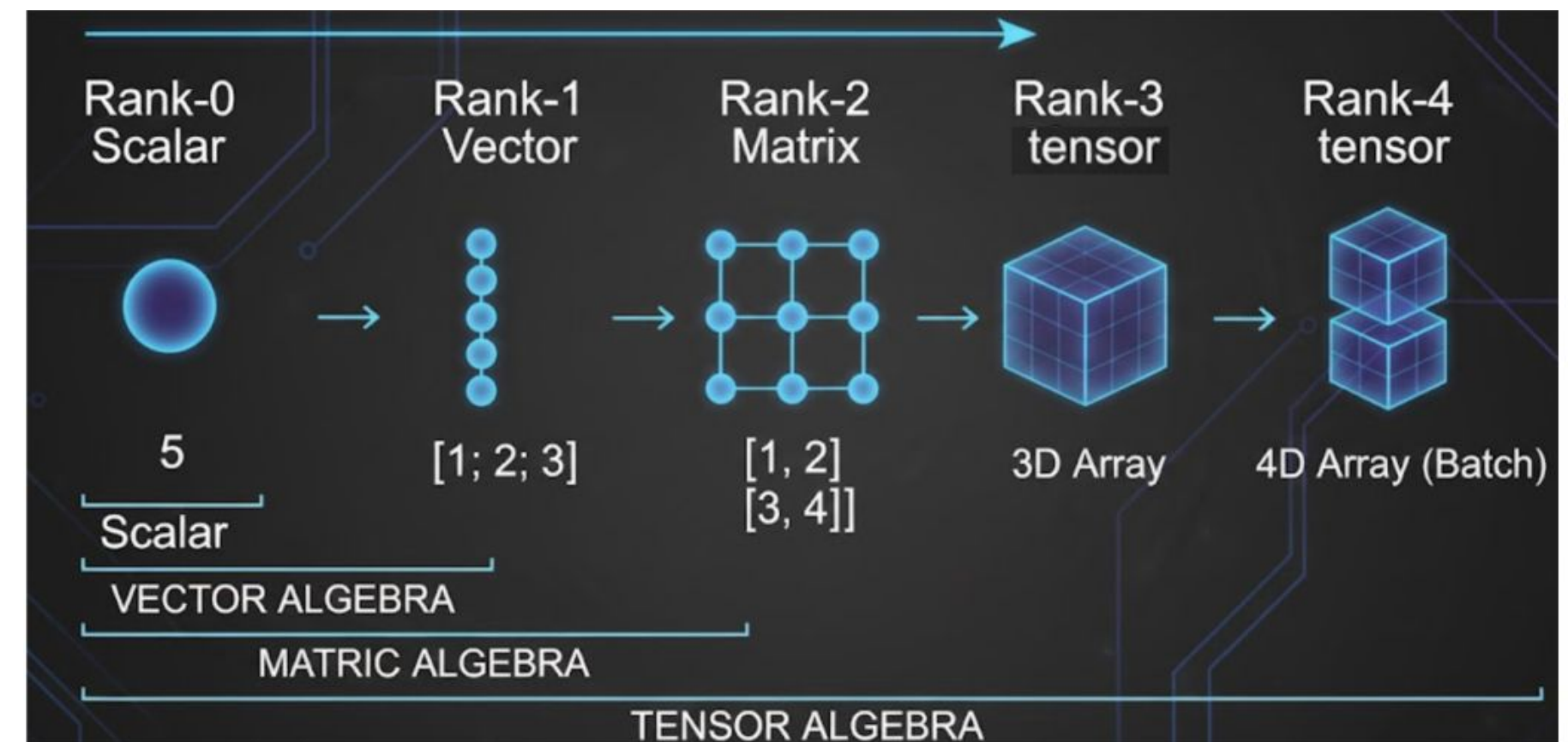
A tensor is a multi-dimensional generalization of matrices.

## Ranks

- Rank-0  $\rightarrow$  Scalar
- Rank-1  $\rightarrow$  Vector
- Rank-2  $\rightarrow$  Matrix
- Rank-3  $\rightarrow$  3D Tensor (e.g., RGB image)

## Deep Learning Relevance

- CNN input = 4D tensor
- Transformer embeddings = 3D tensor
- Audio spectrograms = 3D tensor
- Batch processing = stacking tensors

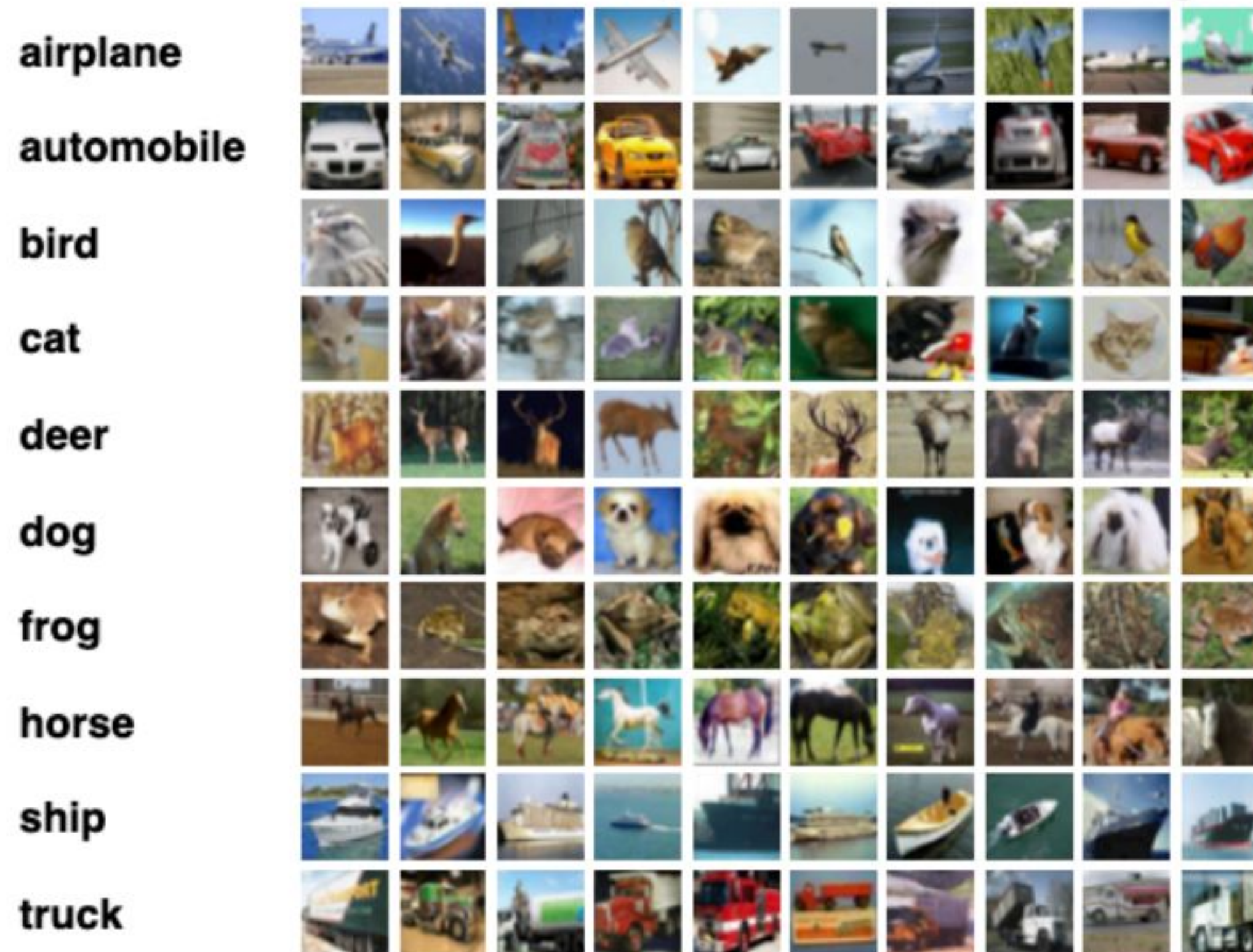


## Analogy

A tensor is like a stack of matrices, just like a stack of notebooks where each notebook page is a matrix.



# Tensors



Batch of images (as neural network input, more later)

# Tensor Operations

## Common Tensor Operations

- Reshape: change tensor shape without changing data
- Transpose: reorder dimensions
- Broadcasting: align shapes
- Reduction: sum, mean across axes

## Real Deep Learning Examples

- CNN expects: (batch, channel, height, width)
- Transformer expects: (batch, seq\_length, model\_dim)
- BatchNorm: computes mean/variance along a specific axis

## Analogy

Think of a tensor as a Rubik's cube:

You can rotate, reshape, and slice it without creating or destroying pieces.



# Matrices

A matrix is a 2D grid of numbers.

Shape: rows  $\times$  columns

Interpretation:

- Rows = observations (students, images, samples)
- Columns = attributes (height, weight, pixel values)

Deep Learning Role

A matrix represents a linear transformation:  $y = Wx$

Where:

- $W$  = weight matrix
- $x$  = input vector
- $y$  = transformed output

Analogy:

Think of a matrix as a recipe book: Each column tells you how strongly to mix the input ingredients.

Scalar    Vector    Matrix

1

$\begin{bmatrix} 1 \\ 2 \end{bmatrix}$

$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$



# Matrices

## Real-Life Examples

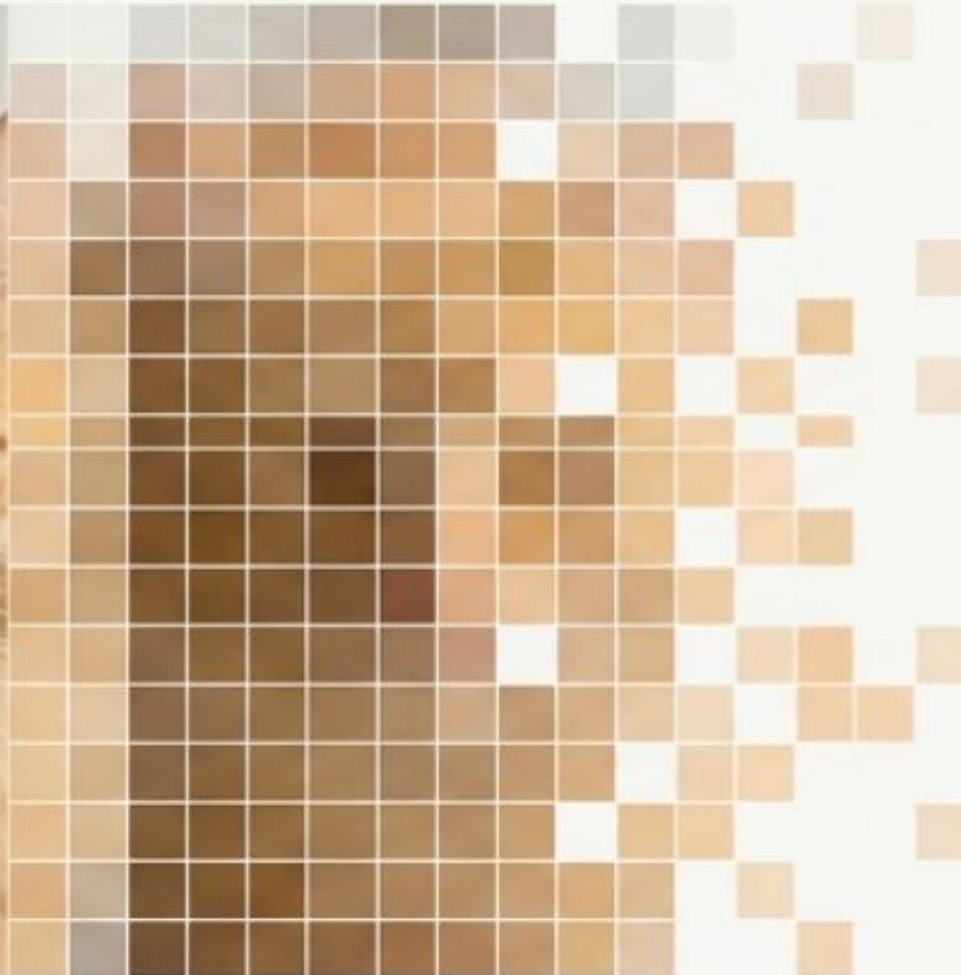
- Classroom gradebook → rows = students, columns = subjects
- Train timetable → rows = stations, columns = timings
- College attendance sheet → matrix of 1s & 0s

## ML Examples

- Image as matrix: grayscale = height × width
- Neural network weights:  $W$
- Transition matrices in RNNs
- Embeddings table (50,000 words × 300 dimensions)

|                         | Mon       | Tue      | Wed      | Thu       |
|-------------------------|-----------|----------|----------|-----------|
| <i><b>Apple</b></i>     | <b>13</b> | <b>9</b> | <b>7</b> | <b>15</b> |
| <i><b>Cherry</b></i>    | <b>8</b>  | <b>7</b> | <b>4</b> | <b>6</b>  |
| <i><b>Blueberry</b></i> | <b>6</b>  | <b>4</b> | <b>0</b> | <b>3</b>  |

# Applications of Matrices



|      |      |      |      |      |      |      |      |      |      |      |
|------|------|------|------|------|------|------|------|------|------|------|
| 0.02 | 0.05 | 0.80 | 0.37 | 0.13 | 0.04 | 0.06 | 0.32 | 0.00 | 0.00 | 0.09 |
| 0.19 | 0.08 | 0.50 | 0.30 | 0.20 | 0.26 | 0.13 | 0.16 | 0.12 | 0.19 | 0.27 |
| 0.72 | 0.11 | 0.48 | 0.25 | 0.15 | 0.31 | 0.34 | 0.35 | 0.24 | 0.15 | 0.28 |
| 0.77 | 0.14 | 0.50 | 1.36 | 0.02 | 0.29 | 0.33 | 0.35 | 0.36 | 0.23 | 0.38 |
| 0.16 | 0.18 | 0.43 | 0.27 | 0.10 | 0.23 | 0.39 | 0.27 | 0.30 | 0.28 | 0.32 |
| 0.27 | 0.28 | 0.45 | 0.38 | 0.13 | 0.18 | 0.18 | 0.30 | 0.23 | 0.23 | 0.31 |
| 0.09 | 0.15 | 0.21 | 0.22 | 0.21 | 0.20 | 0.22 | 0.25 | 0.28 | 0.23 | 0.33 |
| 0.28 | 0.27 | 0.38 | 0.26 | 0.44 | 0.08 | 0.18 | 0.18 | 0.35 | 0.16 | 0.31 |
| 0.24 | 0.34 | 0.42 | 0.26 | 0.36 | 1.00 | 0.36 | 0.44 | 0.57 | 0.22 | 0.28 |
| 0.27 | 0.36 | 0.28 | 0.29 | 0.07 | 0.05 | 1.00 | 0.47 | 0.73 | 0.16 | 0.23 |
| 0.20 | 0.12 | 0.26 | 0.27 | 0.27 | 0.09 | 0.27 | 0.42 | 0.31 | 0.16 | 0.18 |
| 0.16 | 0.17 | 0.28 | 0.19 | 0.14 | 0.14 | 0.16 | 1.00 | 0.31 | 0.13 | 0.26 |
| 0.16 | 0.10 | 0.18 | 0.10 | 0.12 | 0.12 | 0.18 | 1.00 | 0.87 | 0.18 | 0.12 |
| 0.05 | 0.08 | 0.29 | 0.09 | 0.03 | 0.12 | 0.17 | 0.50 | 0.98 | 0.38 | 0.04 |
| 0.13 | 0.08 | 0.07 | 0.08 | 0.06 | 0.25 | 0.13 | 0.19 | 0.37 | 0.43 | 0.00 |
| 0.01 | 0.06 | 0.24 | 0.05 | 0.01 | 0.36 | 0.34 | 0.23 | 0.10 | 0.12 | 0.01 |

Behind every AI decision—every recognized object, every translated sentence—is a series of operations on matrices. This fundamental tool of linear algebra is the universal language for representing and processing information in modern AI.



# Applications of Matrices

## Step 1: Data Representation

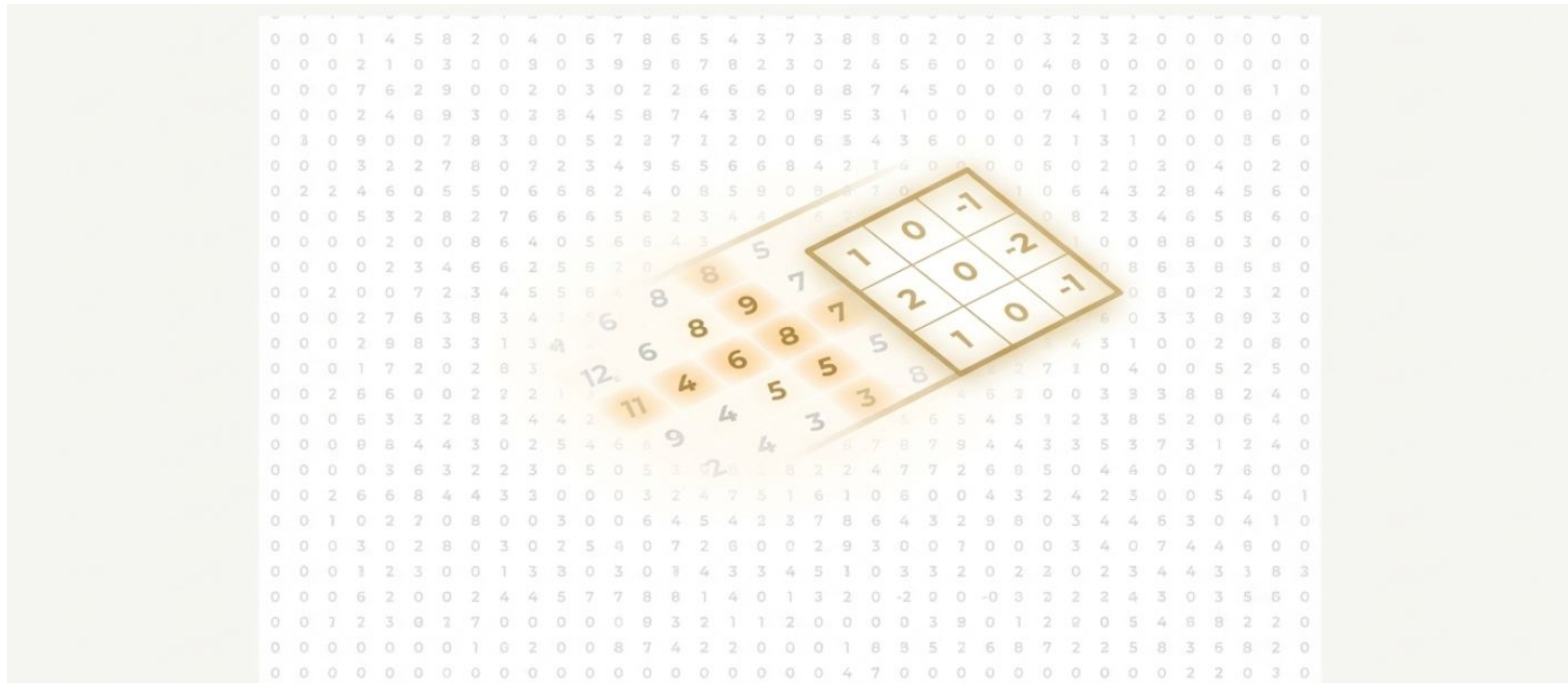


Everything in AI starts as a matrix. A digital image is a matrix of pixel values. A sentence becomes a matrix of numerical vectors called 'word embeddings.' This is the input—the raw material for any AI model.



# Applications of Matrices

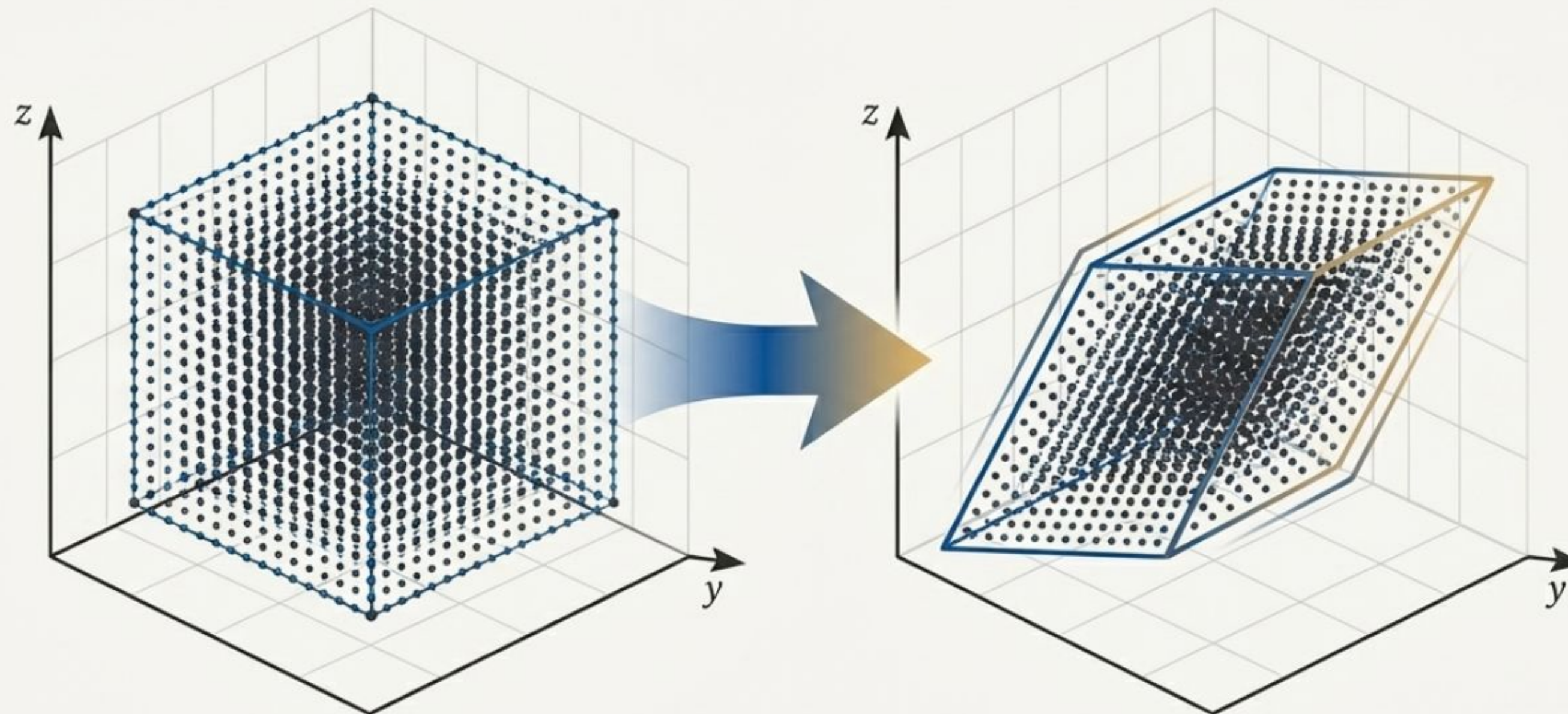
## Step 2 – Convolutional Operations



In computer vision, AI uses small matrices called 'kernels' to find patterns. These filters slide over the input image matrix to detect fundamental features like edges, shapes, and textures. This is how the network begins to build a hierarchical understanding of the visual world.

# Applications of Matrices

## Step 3 – Linear Transformations



Moving data from one layer of a neural network to the next is a linear transformation, performed by matrix multiplication. Each layer “filters” and reshapes the information, extracting more complex patterns from the features identified in the previous step.



# Applications of Matrices

## Step 4 – The Learning Weights

|        |         |         |        |         |        |         |        |         |         |        |
|--------|---------|---------|--------|---------|--------|---------|--------|---------|---------|--------|
| 0.256  | 0.565   | -0.736  | -0.842 | 0.695   | 0.091  | -0.261  | 0.061  | 0.330   | 0.159   | -0.360 |
| -0.284 | 0.666   | 0.382 ↗ | 0.011  | 0.127   | 0.232  | 0.302   | 0.434  | 0.215 ↗ | 0.296 ↗ | 0.013  |
| -0.316 | 0.311 ↗ | -0.190  | -0.124 | 0.253 ↗ | 0.328  | 0.853   | -0.275 | 0.233   | 0.400   | -0.500 |
| -0.563 | 0.767   | -0.907  | -0.285 | 0.37    | 0.202  | 0.797 ↗ | 0.155  | 0.178   | 0.179   | -0.407 |
| -0.284 | 0.013   | -0.684  | 0.507  | 0.557   | -0.164 | -0.595  | 0.17   | 0.096   | 0.303   | -0.365 |
| -0.111 | -0.331  | -0.073  | -0.436 | 0.673 ↗ | 0.549  | 0.663   | -0.386 | 0.149   | 0.204   | -0.187 |
| -0.248 | -0.25   | -0.759  | 0.113  | 0.33    | 0.175  | 1.060 ↗ | -0.014 | -0.331  | 0.136   | 0.251  |
| -0.027 | -0.353  | -0.373  | 0.563  | -0.76   | -0.925 | 0.15    | 0.013  | -0.531  | 0.405   | -0.294 |

When an AI “learns,” it is adjusting the numbers inside massive parameter matrices called “weights.” These weights determine the importance of specific data features. The entire training process is a search for the optimal set of weights that makes the most accurate predictions.



# Matrices

A matrix is a rectangular array of numbers enclosed by a pair of bracket.

$$A = \begin{bmatrix} 2 & 3 & 7 \\ 1 & -1 & 5 \end{bmatrix}$$

In the matrix

$$A = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & & a_{2n} \\ \vdots & & \ddots & \\ a_{m1} & a_{m2} & & a_{mn} \end{bmatrix}$$



- numbers  $a_{ij}$  are called **elements**. First subscript indicates the row, second subscript indicates the column.
- The matrix consists of **mn** elements. It is called “the **m x n** matrix”

# Linear Algebra for Machine Learning

## Data Representation

### Matrix and Vector Operations in Python

#### Scalar

scalar = 1

1

#### Vector

np.array([1,2])

1  
2

#### Matrix

np.array([[1,1],[2,2]])

1 1  
2 2

#### Tensor

np.array([[[1,1],[2,2]], [[3,3],[4,4]]])

1 1 3  
2 2 4

#### Addition

matrix1 + matrix2

1 1  
1 1  
+  
2 2  
2 2  
=  
1+2 1+2  
1+2 1+2

#### Subtraction

matrix2 - matrix1

2 2  
2 2  
-  
1 1  
1 1  
=  
2-1 2-1  
2-1 2-1

#### Multiplication

matrix1 \* matrix2

1 1  
1 1  
x  
2 2  
2 2  
=  
1x2 1x2  
1x2 1x2

#### Division

matrix2 / matrix1

2 2  
2 2  
/  
1 1  
1 1  
=  
2/1 2/1  
2/1 2/1

#### Dot Product

matrix1.dot(matrix2)

1 1  
1 1  
.  
2 2  
2 2  
=  
1x2 1x2  
1x2 1x2

#### Reshape

matrix.reshape(<row>, <col>)

1 2  
2 1  
→  
1  
2  
1  
2

#### Transpose

matrix.T

1 1  
2 2  
3 3  
→  
1 2 3  
1 2 3

#### Inverse

np.linalg.inv(matrix)

identity matrix  
1 0  
0 1  
=  
1 2  
2 1  
·  
-1/3 2/3  
2/3 -1/3

### Applications in Machine Learning

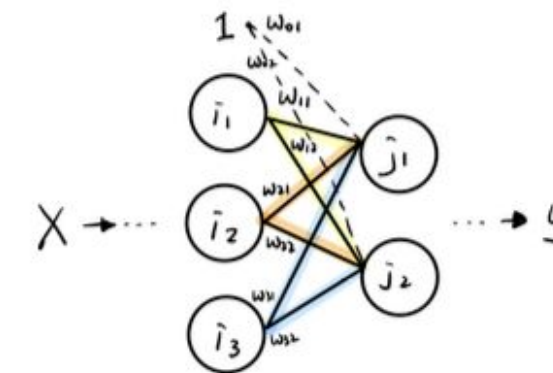
#### Linear Regression



$$[1, x_1, x_2, \dots, x_n] \cdot \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix} = [y]$$

$$y = 1 \cdot w_0 + x_1 w_1 + x_2 w_2 + \dots + x_n w_n$$

#### Neural Networks



$$[1, i_1, i_2, i_3] \cdot \begin{bmatrix} w_{01} & w_{02} \\ w_{11} & w_{12} \\ w_{21} & w_{22} \\ w_{31} & w_{32} \end{bmatrix} = [j_1, j_2]$$

$$j_1 = w_{01} + i_1 w_{11} + i_2 w_{21} + i_3 w_{31}$$

$$j_2 = w_{02} + i_1 w_{12} + i_2 w_{22} + i_3 w_{32}$$

# Order of Matrix

A matrix having m rows and n columns is called a matrix of order m x n or simply m x n matrix.

Determine the size of the matrix shown below?

$$\mathbf{B} = \begin{bmatrix} 2 & 1 \\ 7 & -4 \\ 3 & 1 \end{bmatrix}$$



# Solution

Determine the size of the matrix shown below?

$$\mathbf{B} = \begin{bmatrix} 2 & 1 \\ 7 & -4 \\ 3 & 1 \end{bmatrix}$$

The matrix has 3 rows and 2 columns. Its size is 3 x 2.

# Problem

If a matrix has 8 elements, what are the possible orders it can have?

# Solution

We know that if a matrix is of order  $m \times n$ , it has  $mn$  elements. Thus, to find all possible orders of a matrix with 8 elements, we will find all ordered pairs of natural numbers, whose product is 8.

Thus, all possible ordered pairs are  $(1, 8)$ ,  $(8, 1)$ ,  $(4, 2)$ ,  $(2, 4)$

Hence, possible orders are  $1 \times 8$ ,  $8 \times 1$ ,  $4 \times 2$ ,  $2 \times 4$



## Reshaping a matrix

Original (2, 3)

$$\begin{bmatrix} 1 & 1 & 1 \\ 2 & 2 & 2 \end{bmatrix}$$

(3, 2)

$$\begin{bmatrix} 1 & 2 \\ 2 & 1 \\ 1 & 2 \end{bmatrix}$$

(6, 1)

$$\begin{bmatrix} 1 \\ 2 \\ 1 \\ 2 \\ 1 \\ 2 \end{bmatrix}$$

(1, 6)

$$\begin{bmatrix} 1 & 2 & 1 & 2 & 1 & 2 \end{bmatrix}$$

# Matrix Transpose

- The transpose of a matrix  $A$  is denoted as  $A^T$  and is obtained by interchanging the rows and columns.
- Thus, if  $A$  has a size of  $m \times n$ ,  $A^T$  will have a size of  $n \times m$ .
- If the transpose operation is applied twice, the original matrix is restored.

# Problem

Determine the transpose of the matrix A

$$\mathbf{A} = \begin{bmatrix} 2 & -3 & 5 \\ -1 & 4 & 6 \end{bmatrix}$$

# Solution

$$\mathbf{A} = \begin{bmatrix} 2 & -3 & 5 \\ -1 & 4 & 6 \end{bmatrix}$$

$$\mathbf{A}^T = \begin{bmatrix} 2 & -1 \\ -3 & 4 \\ 5 & 6 \end{bmatrix}$$



# Types of Matrices

- Row matrix
- Column matrix
- Square matrix
- Diagonal matrix
- Scalar matrix
- Identity matrix
- Zero matrix

|                                                                              |                                                                                                       |                                                                                                                        |
|------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| <b>Row Matrix</b><br>$\begin{bmatrix} 3 & 5 & 7 \end{bmatrix}$               | <b>Column Matrix</b><br>$\begin{bmatrix} 2 \\ 4 \\ 6 \end{bmatrix}$                                   | <b>Square Matrix</b><br>$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$                                                 |
| <b>Diagonal Matrix</b><br>$\begin{bmatrix} 4 & 0 \\ 0 & 9 \end{bmatrix}$     | <b>Scalar Matrix</b><br>$\begin{bmatrix} 5 & 0 \\ 0 & 5 \end{bmatrix}$                                | <b>Identity Matrix</b><br>$\begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$                                               |
| <b>Zero or Null Matrix</b><br>$\begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix}$ | <b>Upper Triangular Matrix</b><br>$\begin{bmatrix} 1 & 2 & 3 \\ 0 & 4 & 5 \\ 0 & 0 & 6 \end{bmatrix}$ | <b>Lower Triangular Matrix</b><br>$\begin{bmatrix} 1 & 0 & 0 \\ 2 & 3 & 0 \\ 4 & 5 & 6 \end{bmatrix}$                  |
| <b>Symmetric Matrix</b><br>$\begin{bmatrix} 1 & 2 \\ 2 & 3 \end{bmatrix}$    | <b>Skew-Symmetric Matrix</b><br>$\begin{bmatrix} 0 & 2 \\ -2 & 0 \end{bmatrix}$                       | <b>Singular Matrix</b><br>$\begin{bmatrix} 2 & 4 \\ 1 & 2 \end{bmatrix}$                                               |
| <b>Non-Singular Matrix</b><br>$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ | <b>Orthogonal Matrix</b><br>$\begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix}$                           | <b>Equal Matrices</b><br>$\begin{bmatrix} 2 & 3 \\ 4 & 5 \end{bmatrix} = \begin{bmatrix} 2 & 3 \\ 4 & 5 \end{bmatrix}$ |

# Row Matrix and Column Matrix

A matrix having only one row is called a **row matrix**.

- Here the integer values of time from 0 to 5 sec as a row vector. The size of the vector is 1 x 6 for :  $[0, 1, 2, 3, 4, 5]$

A matrix having only one column is called a **column matrix**.

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

**Note:** The row matrix is called a row vector and the column matrix is called a column vector.

# Square matrix

A matrix in which the number of rows are equal to the number of columns, is said to be a square matrix.

Thus an  $m \times n$  matrix is said to be a square matrix if  $m = n$  and is known as a square matrix of order 'n'.

$$\mathbf{C} = \begin{bmatrix} 2 & -1 & 3 \\ 4 & 6 & 1 \\ -5 & 2 & 1 \end{bmatrix} \text{ is a square matrix of order 3.}$$

# Diagonal matrix

A square matrix is said to be diagonal matrix if all **non-diagonal elements** are zero.

$$A = \begin{bmatrix} -1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 3 \end{bmatrix} \text{ is a diagonal matrix of order 3.}$$



# Scalar matrix

A diagonal matrix is said to be a scalar matrix if its diagonal elements are equal, that square matrix is said to be a scalar matrix.

$$C = \begin{bmatrix} -1 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & -1 \end{bmatrix}$$

# Identity matrix

A square matrix in which elements in the diagonal are all 1 and rest are all zero is called an identity matrix.

$$A=[1], B=\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, C=\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \text{ are}$$

identity matrices of order 1, 2 and 3 respectively

# Zero matrix

A matrix is said to be zero matrix or null matrix if all its elements are zero.

$$C = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

# Symmetric matrix

A matrix  $A$  such that  $A^T = A$  is called symmetric, i.e.,  $a_{ji} = a_{ij}$  for all  $i$  and  $j$ .

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & -5 \\ 3 & -5 & 6 \end{bmatrix}$$



# EQUALITY OF MATRICES

Two matrices  $A = [a_{ij}]$  and  $B = [b_{ij}]$  are said to be equal if:

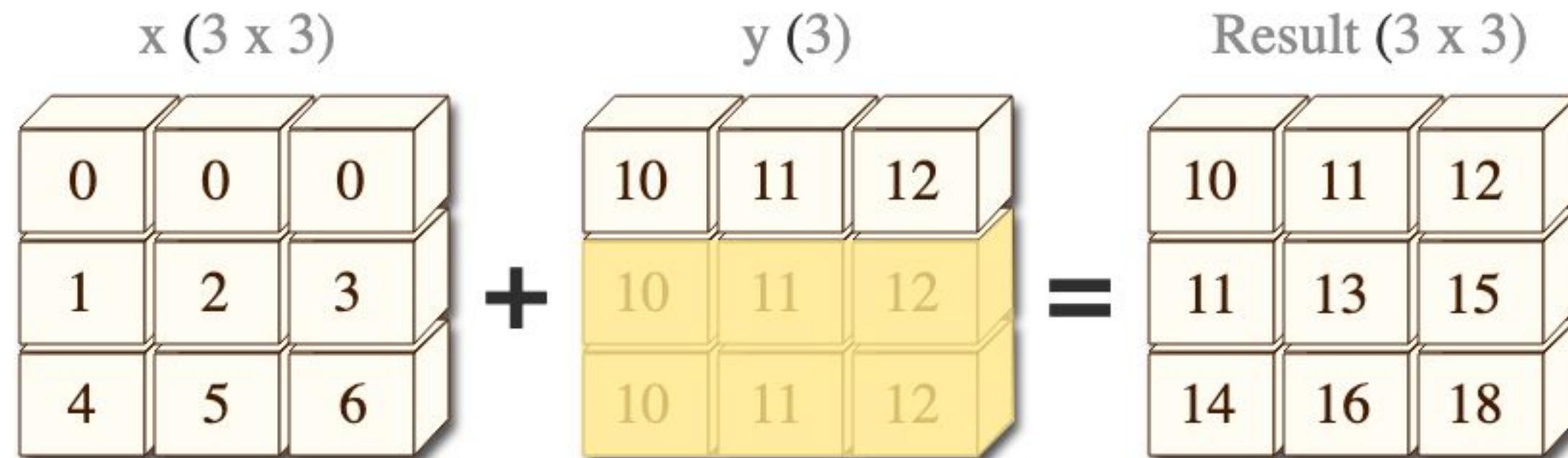
( i ) they are of the same order

( ii ) each element of A is equal to the corresponding

Symbolically if two matrices are equal we write  $A=B$ .

# Broadcasting: Making Shapes Compatible

Broadcasting allows operations between arrays of different shapes by automatically expanding the smaller one.



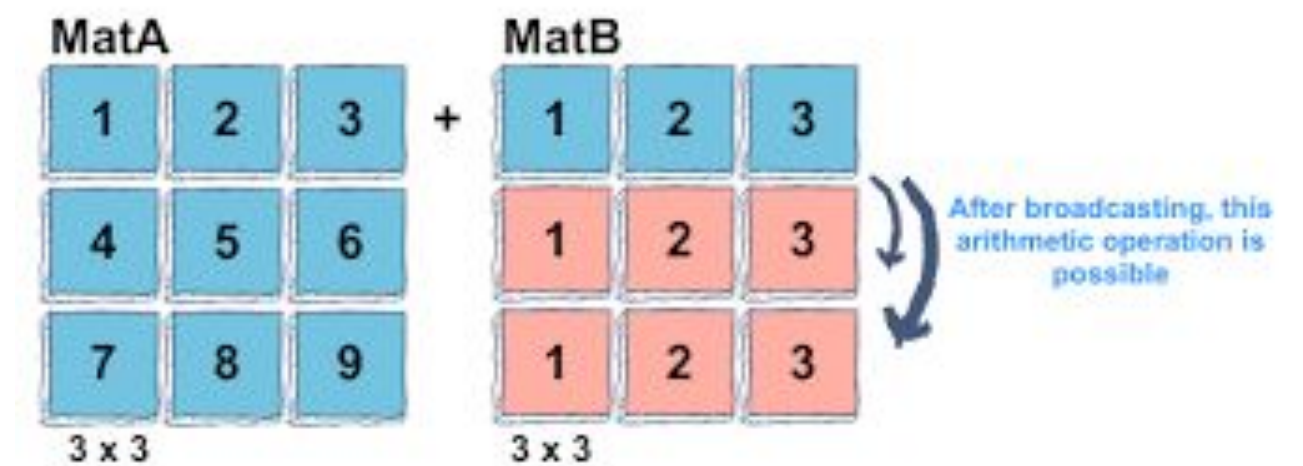
# Broadcasting: Making Shapes Compatible

## Why It Matters in ML

- Adding bias to neural layers  $\rightarrow y = Wx + b$
- Normalizing data (subtract mean vector)
- Feature scaling

## Analogy

Think of broadcasting like copy-pasting one row to match the height of the matrix.



## Practice Question

A is  $5 \times 4$ , b is  $1 \times 4$ . Can we do  $A + b$ ? Explain why.

# Matrix Multiplication

General Rule:

If A is  $m \times n$  and B is  $n \times p$ , then AB is  $m \times p$ .

Each element of AB is:  $c_{ij} = \sum_k (a_{ik} \times b_{kj})$

| Matrix 1 |   |
|----------|---|
| 1        | 1 |
| 2        | 2 |
| 3        | 3 |

(3x2)

×

| Matrix 2 |   |   |
|----------|---|---|
| 1        | 1 | 1 |
| 2        | 2 | 2 |

(2x3)

=

| Product matrix |   |   |
|----------------|---|---|
| 3              | 3 | 3 |
| 6              | 6 | 6 |
| 9              | 9 | 9 |

(3x3)



# Matrix Multiplication

## Relatable Example

Consider:

$A = \text{students} \times \text{features}$

$B = \text{features} \times \text{exam weightage}$

Then  $AB = \text{students} \times \text{exam score}$

This is literally how weighted scores are computed in college.

## Visual Intuition

Row of  $A$  “mixes” with column of  $B$ . Each output value is dot product of row & column

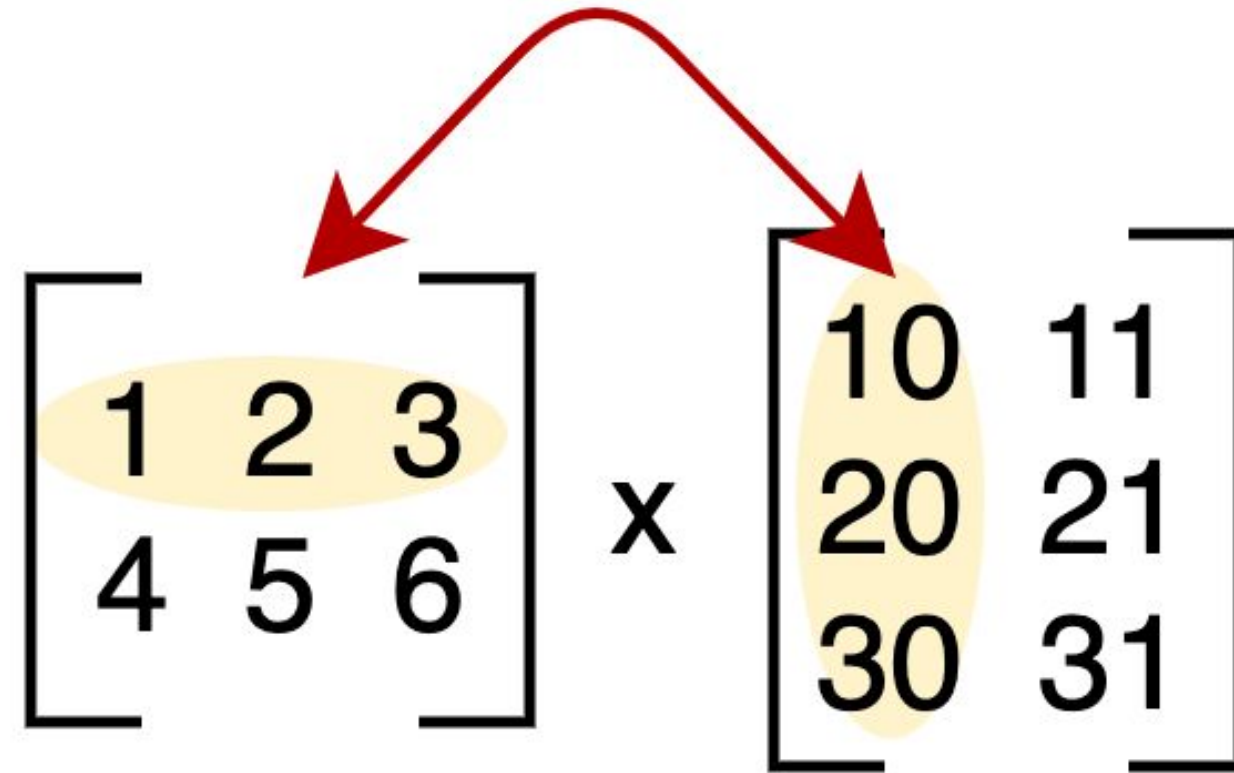
Note: Matrix multiplication is **not commutative**

## Practice Question

If  $A$  is  $4 \times 7$  and  $B$  is  $7 \times 2$ , what will be the shape of  $AB$ ?

# Problem

Compute  $A*B$


$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \times \begin{bmatrix} 10 & 11 \\ 20 & 21 \\ 30 & 31 \end{bmatrix}$$

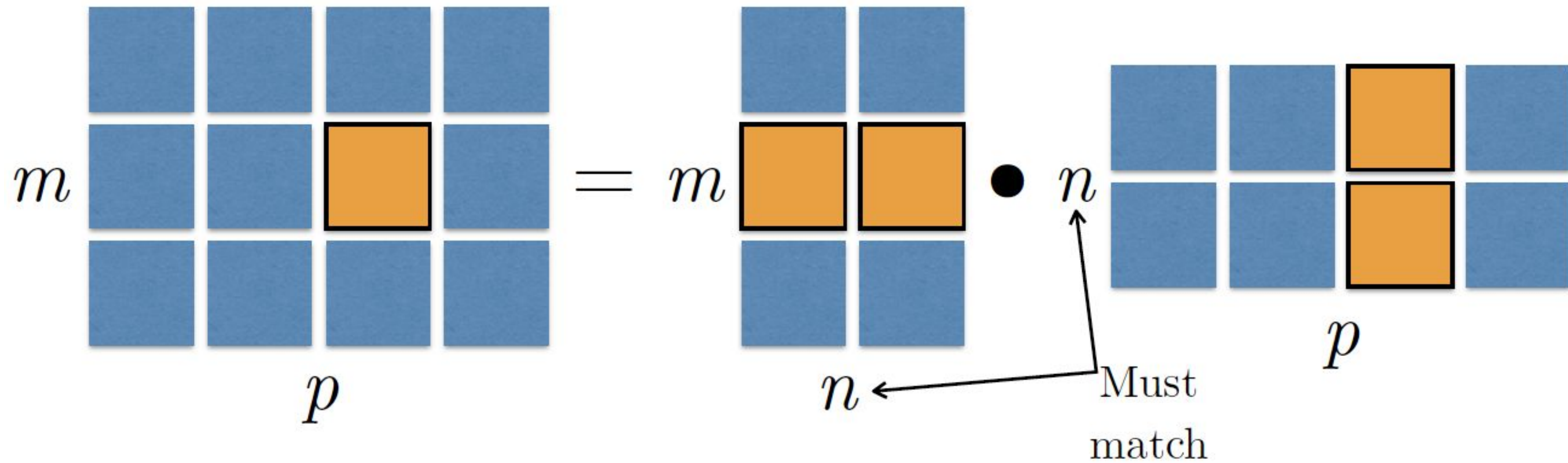
# Problem

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix} \times \begin{bmatrix} 10 & 11 \\ 20 & 21 \\ 30 & 31 \end{bmatrix}$$
$$= \begin{bmatrix} 1 \times 10 + 2 \times 20 + 3 \times 30 & 1 \times 11 + 2 \times 21 + 3 \times 31 \\ 4 \times 10 + 5 \times 20 + 6 \times 30 & 4 \times 11 + 5 \times 21 + 6 \times 31 \end{bmatrix}$$
$$= \begin{bmatrix} 10+40+90 & 11+42+93 \\ 40+100+180 & 44+105+186 \end{bmatrix} = \begin{bmatrix} 140 & 146 \\ 320 & 335 \end{bmatrix}$$

# Matrix Dot Product

$$C = AB.$$

$$C_{i,j} = \sum_k A_{i,k} B_{k,j}.$$



# Identity Matrix

Consider the nxn matrix:

For any nxn matrix A, we have :  $AI_n = I_n A = A$

For any nxm matrix A, we have :  $I_n A = A$   
and  $AI_m = A$

$$I_n = \begin{pmatrix} 1 & 0 & 0 & \dots & 0 \\ 0 & 1 & 0 & \dots & 0 \\ 0 & 0 & 1 & \dots & 0 \\ \vdots & & & & \vdots \\ 0 & 0 & 0 & \dots & 1 \end{pmatrix}.$$



# Example

For a 3x3 matrix:

$$\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix} \times \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1+0+0 & 0+2+0 & 0+0+3 \\ 4+0+0 & 0+5+0 & 0+0+6 \\ 7+0+0 & 0+8+0 & 0+0+9 \end{bmatrix}$$

$$\mathbf{A} \mathbf{I}_3 = \mathbf{A}$$

# Matrix Properties

1. Associativity :  $A(BC) = (AB)C$
2. Distributivity :  $A(B+C) = AB + AC$
3. Scalar multiplication :  $A(BC) = (AB)C = B(AC)$
4. Transpose of a product:  $(AB)^T = B^T A^T$

# Matrix Properties

## Why ML/DL Cares

- Backpropagation uses transpose and distributive rules
- Simplifies gradient derivations
- Allows efficient computation in deep models

Common Misconception:

**$AB \neq BA$  (NOT commutative)**

# Determinants

Determinant of order 2:

Consider a 2 x 2 matrix:

$$A = \begin{bmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{bmatrix}$$

Determinant of A, can be evaluated by:

$$|A| = \begin{vmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{vmatrix} = a_{11}a_{22} - a_{12}a_{21}$$

# Determinants

Determinant of order 3:

Consider a 3 x 3 matrix:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Its determinant can be obtained by:

$$\begin{aligned} |A| &= \begin{vmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{vmatrix} = 3 \begin{vmatrix} 4 & 5 \\ 7 & 8 \end{vmatrix} - 6 \begin{vmatrix} 1 & 2 \\ 7 & 8 \end{vmatrix} + 9 \begin{vmatrix} 1 & 2 \\ 4 & 5 \end{vmatrix} \\ &= 3(-3) - 6(-6) + 9(-3) = 0 \end{aligned}$$



# PROBLEM

Find the value of

$$\text{i) } \begin{vmatrix} 3 & -5 & 4 \\ 7 & 6 & 1 \\ 1 & 2 & 3 \end{vmatrix}$$

$$\text{ii) } \begin{vmatrix} 1 & 4 & 7 \\ -2 & 3 & 4 \\ 1 & 4 & -4 \end{vmatrix}$$

# Determinants

The following properties are true for determinants of any order.

1. If every element of a row (column) is zero, then  $|A| = 0$
2. determinant of a matrix is equal to its transpose  $|A^T| = |A|$

*(There are more properties for special cases such as  $|AB| = |A||B|$  )*

# Matrix Inverse

A matrix A is called nonsingular or invertible if there exists a matrix B such that:

$$A B = B A = I_n$$

$$\begin{bmatrix} 1 & 1 \\ -1 & 2 \end{bmatrix} \times \begin{bmatrix} \frac{2}{3} & \frac{-1}{3} \\ \frac{1}{3} & \frac{1}{3} \end{bmatrix} = \begin{bmatrix} \frac{2}{3} + \frac{1}{3} & \frac{-1}{3} + \frac{1}{3} \\ \frac{-2}{3} + \frac{2}{3} & \frac{1}{3} + \frac{2}{3} \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

A common notation for the inverse of a matrix A is  $A^{-1}$ .

# Matrix Inverse

- A matrix  $A$  is called nonsingular or invertible if there exists a matrix  $B$  such that:  $(A^T)^{-1} = (A^{-1})^T$
- Matrix division:  $A/B = A * B^{-1}$

# Matrix Inverse

- The inverse matrix is:

$$A^{-1} = \frac{1}{\det(A)} \begin{pmatrix} \text{cof}(A, x_{1,1}) & \dots & \text{cof}(A, x_{1,j}) \\ \vdots & \ddots & \vdots \\ \text{cof}(A, x_{i,1}) & \dots & \text{cof}(A, x_{i,j}) \end{pmatrix}^T$$

- Example for 2X2 matrix:

$$\mathbf{A}^{-1} = \begin{bmatrix} a & b \\ c & d \end{bmatrix}^{-1} = \frac{1}{ad - bc} \begin{bmatrix} d & -b \\ -c & a \end{bmatrix}$$



# Norms in Geometry:

## The Geometry of Learning: Vector Norms

In Deep Learning, norms tell us the “shape” of distance. They are the mathematical rulers we use to measure the size of weights and gradients.

**L1 Norm (“Manhattan Distance”)**

**L2 Norm (“Euclidean Distance”)**

**Max Norm (“Infinity Norm”)**

# Norms in Geometry:

## L1 Norm (“Manhattan Distance”)

- Shape in 2D: → Diamond
- Intuition: You can move only in horizontal/vertical paths
- Math: Sum of absolute distances
- DL Effect: Promotes Sparsity  
(encourages many weights to become exactly zero)

# Norms in Geometry:

## L2 Norm (“Euclidean Distance”)

- Shape in 2D:  $\rightarrow$  Circle
- Intuition: Straight-line distance between points
- Math: Most commonly used
- DL Effect: Isotropic (Uniform); keeps weights small and spread out

# Norms in Geometry:

## Max Norm ("Infinity Norm")

- Shape in 2D: → Square
- Intuition: The "Absolute Maximum" path; it only cares about the single largest value in the vector.
- DL Effect: Acts as a "Hard Ceiling."  
Used in Max-Norm Regularization to prevent weights from growing past a specific boundary, which helps stabilize training against exploding gradients.

- Math: 
$$L_{\infty} \quad ||x||_{\infty} = \max(|x_1|, |x_2|, \dots, |x_n|)$$

# How Norms Control Learning (ML Applications)

## L2 Regularization (Weight Decay)

- Penalizes large weights
- Encourages smooth models
- Prevents overfitting

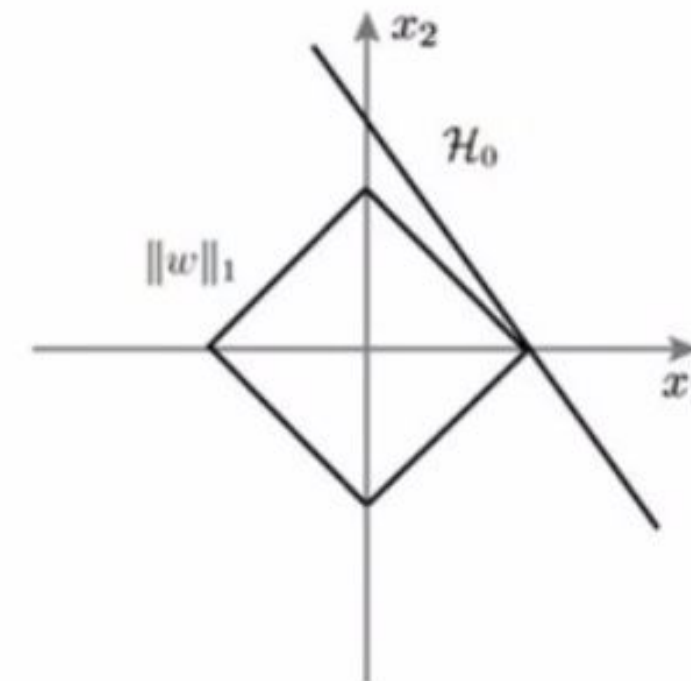
## L1 Regularization

- Encourages sparse models
- Useful for feature selection

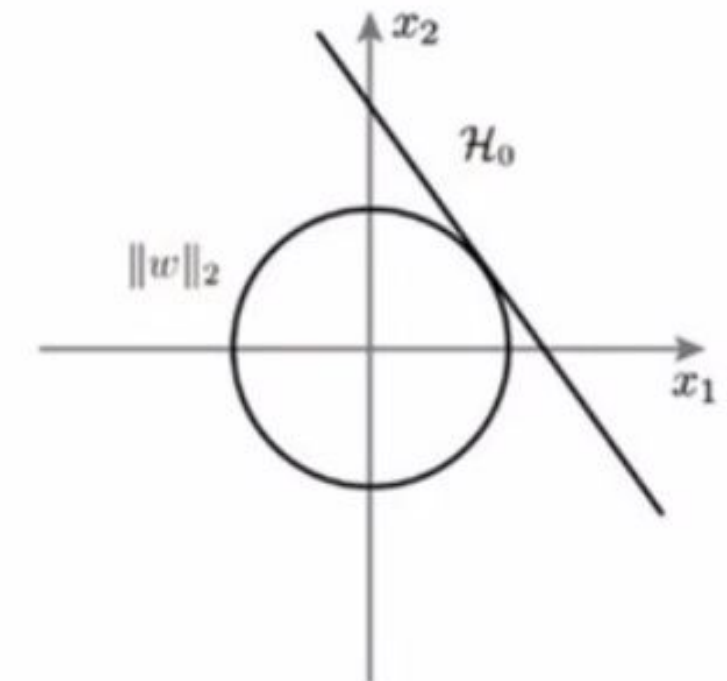
## Gradient Norms

- Used to detect exploding/vanishing gradients
- Used in gradient clipping (L2 norm)

**A** L1 regularization



**B** L2 regularization





# How Norms Control Learning (ML Applications)

## Sparsity vs. Stability

- **Lasso (L1):** Adds the L1 norm to the loss. It acts like a "sieve," zeroing out unimportant features. Great for interpretability.
- **Ridge (L2):** Adds the squared L2 norm. It shrinks all weights towards zero but rarely hits zero. Great for stability and preventing over-reliance on one feature.

# 5-Minute Quiz (Concept + Computation)

## Conceptual Questions

1. What is the difference between a vector and a tensor?
2. Why can matrix multiplication not be commutative?

## Computational Questions

3. Compute the dot product:  $x = [1,2,3]$ ,  $y = [4,5,6]$
4. Find the shape of AB:  
If A is  $3 \times 4$  and B is  $4 \times 2$  Compute norms:  $\| [3,4] \|_2 = ?$ ,  $\| [3,4] \|_1 = ?$

## Deep Learning Thinking

5. If  $\delta$  is  $3 \times 1$  and  $x$  is  $5 \times 1$ , what is the shape of  $\delta x^T$ ?
6. Why does CNN convolution require tensors instead of matrices?